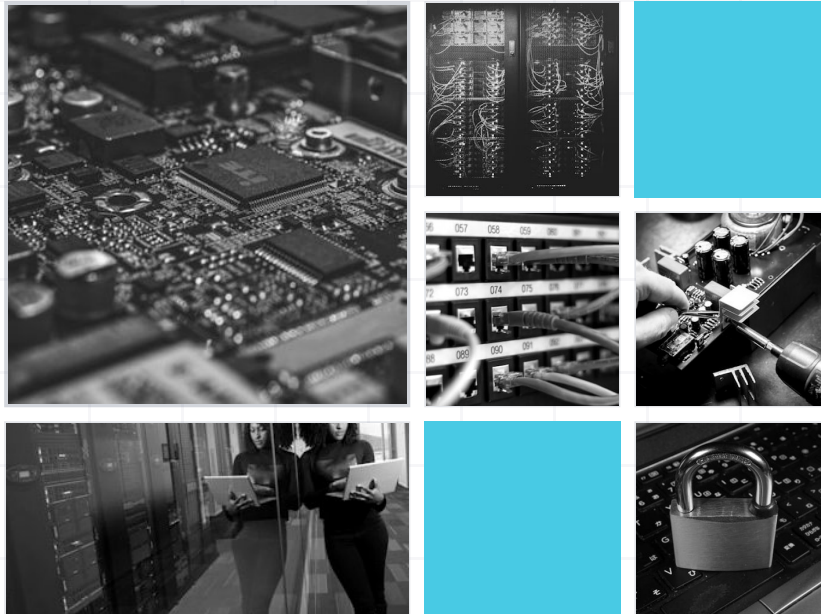




DOKUMENTASI TEKNIS

FALCON FPGA GTP-U DPI

Dari Nol sampai Berjalan + Roadmap + Glosarium

Disiapkan oleh **Arah Karya Sinergi (AKS)**

2026

NOZ × Arah Karya Sinergi
Fase 1.5 — Terverifikasi

AKS · v1.1

Dokumentasi Teknis FALCON

Proyek	FALCON — FPGA-Accelerated Live Core Observation Node
Kolaborasi	NOZ × Arah Karya Sinergi (AKS)
Dokumen	Laporan Teknis: Dari Nol sampai Berjalan + Rencana Ke Depan
Versi	1.0
Tanggal	Juni 2026
Status	Fase 1.5 — TUNTAS, terverifikasi end-to-end pada silikon nyata

1. Ringkasan Eksekutif

FALCON adalah sistem *monitoring* trafik jaringan seluler (4G/5G) yang men-*parse* paket **GTP-U** secara *real-time* menggunakan akselerator **FPGA**, lalu menampilkan hasilnya sebagai *dashboard* web yang hidup. Inti gagasannya: pekerjaan parsing paket yang berat dikerjakan oleh *gateway* di dalam chip FPGA (bukan CPU), sehingga sistem mampu mengamati trafik pada *line-rate* tanpa membebani prosesor host.

Dokumen ini merekam perjalanan teknis proyek **dari awal pengembangan hingga sistem benar-benar berjalan**, termasuk satu *bug* kritis yang sempat menghalangi seluruh pipeline dan bagaimana akar masalahnya ditemukan, serta **rencana pengembangan ke depan**. Di bagian akhir disertakan **glosarium** istilah teknis agar dokumen dapat dibaca oleh pembaca non-spesialis.

Capaian utama (Fase 1.5): keempat tipe datagram telemetri (GLOBAL, PER-TEID, EVENT, PROTOCOL) berhasil dipancarkan dari FPGA fisik (Virtex-5 **XC5VLX50T**), diterima backend, dan dirender di dashboard secara *live* dengan **nol parse error**. Seluruh rantai end-to-end telah diverifikasi dengan mata.

2. Apa Itu FALCON (Konsep)

Bayangkan jaringan operator seluler sebagai jalan tol yang dilalui jutaan kendaraan (paket data) per detik. Untuk mengetahui apa yang lewat — siapa, berapa banyak, jenis trafik apa — biasanya dibutuhkan komputer yang memeriksa setiap kendaraan satu per satu. Pada kecepatan tol modern, satu komputer (CPU) **tidak sanggup** memeriksa semuanya tanpa tertinggal.

FALCON memindahkan tugas pemeriksaan itu ke **FPGA** — chip yang dapat "dibentuk ulang" menjadi sirkuit khusus. Sirkuit ini memeriksa paket **secara paralel di tingkat perangkat keras**, jauh lebih cepat daripada perangkat lunak. Hasil pemeriksaan diringkas menjadi pesan-pesan kecil (datagram telemetri) dan dikirim ke komputer host hanya untuk ditampilkan — bukan untuk diproses berat.

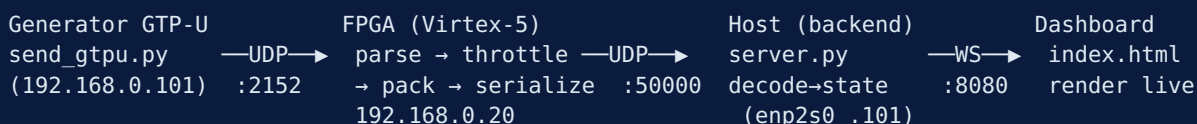
Pembagian peran:

Komponen	Tugas	Mengapa di situ
FPGA (gateway)	Parse GTP-U, hitung statistik, klasifikasi protokol	Cepat, paralel, hemat CPU
Host (backend)	Terima telemetri, decode, simpan state, sajikan	Fleksibel, mudah diubah

Komponen	Tugas	Mengapa di situ
Dashboard (web)	Tampilkan data live ke manusia	Visualisasi & inspeksi

3. Arsitektur Sistem

3.1 Alur Data End-to-End



1. **Generator** mengirim paket GTP-U sintetik ke board FPGA (`192.168.0.20:2152`).
2. **FPGA** mem-*parse* tiap paket, mengekstrak TEID, menghitung statistik, mengklasifikasi protokol, lalu **men-throttle** (memilih sebagian sampel agar tidak membanjiri jaringan), mengemas (*pack*) menjadi datagram, dan men-*serialize* ke kabel sebagai UDP.
3. Datagram telemetry dikirim ke host (`192.168.0.101:50000`).
4. **Backend** men-*decode* tiap datagram memakai **kontrak byte** bersama, memperbarui state, lalu mem-*broadcast* ke dashboard via **WebSocket**.
5. **Dashboard** menampilkan data secara *real-time*.

3.2 Empat Tipe Datagram Telemetry

Inti kontrak FALCON: setiap pesan diawali **header 4-byte** (`msg_type` , `version` , `length`) lalu *payload*. Empat tipe:

ID	Tipe	Payload	Isi
<code>0x01</code>	GLOBAL	64 B	Statistik agregat: total IMSI, uplink/downlink pps, jumlah TEID aktif, total byte, drop
<code>0x02</code>	PER-TEID	48 B	Per-sesi: TEID, IMSI, QFI, state (ACTIVE/IDLE), paket UL/DL
<code>0x03</code>	EVENT	32 B	Kejadian: CreateSession / ModifySession / DeleteSession / Error, arah, TEID
<code>0x04</code>	PROTOCOL	32 B	Distribusi protokol: gtp_u, gtp_c, pfcpl, bssgpl, other (persen)

Prinsip desain kunci — kontrak byte tunggal. Struktur byte ini didefinisikan satu kali (`shared/contract.py`) dan dipakai bersama oleh sumber data dan backend. Akibatnya, selama board FPGA masih dikembangkan, sebuah **Simulator** dapat menghasilkan telemetry dengan kontrak byte **identik** — seluruh stack host bisa dibangun & diuji **tanpa hardware**. Saat board siap: matikan simulator, arahkan FPGA ke `:50000` , **nol baris kode host berubah**.

4. Perjalanan Pengembangan: Dari Nol sampai Berjalan

4.1 Tahap Fondasi (host-first)

Karena board FPGA dikembangkan paralel di pihak NOZ, strategi yang diambil adalah **host-first**: bangun backend + dashboard lebih dulu, diberi makan oleh **Simulator** yang meniru kontrak byte FPGA. Ini memungkinkan seluruh perangkat lunak matang sebelum silikon tersedia — mengurangi risiko integrasi.

Hasil tahap ini: backend asyncio (UDP collector + WebSocket push + REST API), dashboard web bertema *Hermes Dashboard* (navy + teal, flat, monospace), dan kontrak byte yang stabil.

4.2 Tahap Integrasi Silikon (Fase 1.5)

Setelah board FPGA Virtex-5 tersedia, target Fase 1.5 ditetapkan: **board harus memancarkan keempat tipe datagram**, bukan hanya GLOBAL. Di sinilah muncul *bug* yang paling menantang sepanjang proyek.

Gejala

Hanya datagram **GLOBAL** yang sampai ke jaringan, kira-kira **1 paket per detik**. Datagram TEID, EVENT, dan PROTOCOL **tidak pernah muncul** di host — padahal secara logika gateway seharusnya menghasilkannya.

Investigasi (13+ ronde rebuild)

Berbagai hipotesis dikejar dan dibuktikan **tidak bersalah** satu per satu: sinyal yang ter-trim oleh sintesis, batas bahasa campuran (Verilog/VHDL), pengaturan *throttle*, dan logika *arbiter*. Untuk berhenti menebak, dipasang **probe instrumentasi** — penghitung internal (`thr_cnt` , `dbg_teid_cnt` , `sof_cnt` , dst.) yang nilainya "dititipkan" ke field datagram GLOBAL yang masih bisa lewat. Probe membuktikan rantai internal **hidup** (`teidfire=639` , `pdone≈41594`) — tetapi hanya 7 datagram yang benar-benar sampai ke jaringan.

Akar masalah

Satu baris di `fpga/eth/fpga_core.v` : `tx_udp_length` **di-hardcode** `8 + 68` . Nilai 68 adalah panjang payload GLOBAL. Untuk GLOBAL, panjang ini benar sehingga paket lolos. Tetapi TEID (52 B), EVENT & PROTOCOL (36 B) **panjangnya berbeda** — ketidakcocokan panjang membuat core UDP keluar dari sinkronisasi (*desync*) sekitar 1 detik, sehingga hanya GLOBAL @1Hz yang bocor keluar.

Pelajaran: RTL FALCON sebenarnya **selalu benar**. 13 rebuild mengejar hantu. Instrumentasi probe yang **terarah** (bukan tebakan buta) yang akhirnya mengisolasi satu baris penyebab.

Tiga perbaikan permanen

1. `tx_udp_length = 8 + w_tx_len` — panjang UDP dihitung per-tipe (port `tx_len_o` dari wrapper VHDL diekspos & disambungkan).

2. **Gerbang** `hdr_done` (**header-first**) — payload hanya dikirim setelah header UDP valid & diterima, mencegah paket cacat.
3. **Parser** `udp_dport` **jadi konstanta kombinatorial** — mematikan *trim* sinyal yang keliru oleh sintesis (XST).

Bonus: *refactor* seluruh jalur ke **VHDL** menghilangkan batas bahasa campuran.

Hasil setelah perbaikan

Histogram bersih dari board: `{GLOBAL: 8, TEID: 624, EVENT: 625, PROTOCOL: 8}` — sekitar **180 datagram/detik, keempat tipe mengalir**, kontrak bersih.

4.3 Verifikasi End-to-End

Injeksi 60.000 paket diuji menembus seluruh rantai. Backend melaporkan: `msg_count` naik konsisten, `err_count = 0`, `uplink_pps = 180000`, decode EVENT membawa TEID nyata (mis. `0xB0A04DC4`), dan tabel TEID terisi 12 sesi `ACTIVE` saat injeksi berjalan (kosong saat idle — perilaku benar, sesi *expire* karena TTL).

4.4 Connection Layer — FPGA Telemetry Log

Sebagai pelengkap observabilitas, ditambahkan **panel "FPGA Telemetry Log"** di dashboard: *feed* datagram **mentah** yang dipancarkan FPGA secara *live* — menampilkan waktu (presisi milidetik), tipe (badge berwarna), panjang, TEID ter-*decode*, dan **16 byte awal dalam heksadesimal**. Backend menyimpan *ring buffer* 200 datagram terakhir, menyajikannya via `GET /api/rawlog` dan men-*push* tiap datagram lewat WebSocket. Verifikasi byte mentah cocok dengan kontrak (`02 01 00 30 ...` = TEID v1 panjang 48, dst.). Banner dashboard juga diperbarui dari "**SIMULASI**" menjadi "**● LIVE · FPGA**" karena data kini berasal dari silikon nyata.

5. Cara Memverifikasi FPGA Berjalan

Empat lapis pemeriksaan, dari paling dekat ke silikon hingga data sampai layar:

Lapis	Yang dicek	Perintah	Indikator sehat
1. Link fisik	Board ↔ host hidup	<code>ip -br addr show enp2s0 + ip neigh 192.168.0.20</code>	Interface UP, ARP <code>02:00:00:00:00:20</code>
2. Telemetri raw	Board memancar UDP :50000	<code>sudo python3 sniff_types.py</code>	Histogram keempat tipe
3. Backend health ☆	<code>msg_count</code> bertambah	<code>curl :8080/api/health</code> (2×)	<code>msg_count</code> naik, <code>err_count = 0</code>
4. Snapshot decode	TEID nyata ter-decode	<code>curl :8080/api/snapshot</code>	TEID real, state ACTIVE

Cek tercepat & paling andal — Lapis 3. Panggil `/api/health` dua kali berjeda 3 detik. Jika `msg_count` **naik** → FPGA memancar (hidup). Jika **diam** → FPGA mati/tidak memancar. Jika `err_count > 0` → ada datagram cacat (ketidakcocokan kontrak).

Catatan operasional penting: setiap kali board di-*flash* ulang (`xc3sprog -c xpc -v <bitfile>`), **IP interface host (`enp2s0`) ikut hilang** dan harus di-set ulang ke `192.168.0.101` . Dashboard hanya menampilkan data *live* saat ada injeksi trafik — saat idle, tabel TEID menampilkan "menunggu data..." (normal).

6. Status Saat Ini

Aspek	Status
Emit 4 tipe datagram dari silikon	✓ Tuntas
Pipeline end-to-end (gen → FPGA → host → backend → dashboard)	✓ Terverifikasi
Parse error	✓ Nol (<code>err_count = 0</code>)
Dashboard live via Tailscale	✓ Dapat diakses (<code>http://lab:8080</code>)
Panel FPGA Telemetry Log (raw feed)	✓ Live
Repositori (GitHub ArahKarya/falcon)	✓ Tersinkron

Diketahui (bukan bug Fase 1.5, masuk fase lanjut):

- **IMSI = 0** di semua TEID — gateway belum mengekstrak IMSI dari paket (field IMSI masih kosong).
- **Event type = 255** — saat ini memakai *message-type* GTP-U mentah; pemetaan ke CreateSession/Modify/Delete belum diterapkan.

7. Rencana Ke Depan (Roadmap)

Fase 2 — Ekstraksi Identitas & Sesi

- **Ekstraksi IMSI** dari paket di gateway → tabel sesi lengkap (TEID ↔ IMSI).
- **Pemetaan event** GTP-U message-type ke semantik sesi (CreateSession/ModifySession/DeleteSession).
- **State machine sesi** di FPGA: IDLE → ACTIVE → SUSPENDED yang akurat.

Fase 3 — Cakupan Protokol & Arah

- Dukungan **downlink** (saat ini fokus uplink).
- Klasifikasi protokol lebih kaya: **GTP-C, PFCP, BSSGP** (bukan hanya GTP-U).
- Penghitungan throughput byte akurat (saat ini `total_bytes` belum terisi penuh).

Fase 4 — Keandalan & Skala

- **Throttle adaptif** mengikuti beban, bukan rasio tetap.
- **Buffering & back-pressure** untuk menghindari drop saat puncak trafik.
- **Persistensi**: simpan riwayat telemetry (time-series) untuk analitik historis.

Fase 5 — Produk & Operasi

- **Autentikasi & multi-tenant** pada dashboard (saat ini single-view).
- **Alarm & ambang** (notifikasi saat drop/error/anomali melewati batas).
- **Deployment** terkelola (akses publik via Cloudflare tunnel — falcon.arahkarya.com).
- **Privasi & compliance** — tokenisasi IMSI, data minimization, retensi & audit (wajib untuk data telco nyata).

Prinsip lintas-fase: kontrak byte tunggal dijaga sebagai *source of truth*. Perubahan internal gateway tidak boleh memaksa perubahan besar di host — header `length` membuat dampak perubahan minimal.

8. Glosarium Istilah Teknis

Istilah	Pengertian
FPGA	<i>Field-Programmable Gate Array</i> — chip yang sirkuit internalnya dapat "dibentuk ulang" oleh perancang menjadi rangkaian digital khusus. Memungkinkan pemrosesan paralel di tingkat perangkat keras, jauh lebih cepat dari perangkat lunak untuk tugas tertentu.
Gateway	Istilah untuk "perangkat lunak" yang berjalan di FPGA. Bukan program biasa, melainkan deskripsi sirkuit. Analog dengan <i>firmware</i> , tetapi spesifik FPGA.
Virtex-5 / XC5VLX50T	Keluarga & tipe chip FPGA buatan Xilinx yang dipakai pada board FALCON.
RTL	<i>Register-Transfer Level</i> — cara mendeskripsikan sirkuit digital sebagai aliran data antar register. Ditulis dalam bahasa HDL.
HDL / VHDL / Verilog	<i>Hardware Description Language</i> — bahasa untuk mendeskripsikan sirkuit. VHDL dan Verilog adalah dua HDL paling umum. FALCON kini seluruhnya VHDL.
Sintesis (Synthesis) / XST	Proses mengubah kode HDL menjadi sirkuit nyata di FPGA. XST adalah alat sintesis Xilinx. Sintesis dapat membuang (<i>trim</i>) sinyal yang dianggap tak terpakai — sumber salah satu <i>bug</i> .
Bitstream	Berkas hasil akhir sintesis yang "ditanamkan" (<i>flash</i>) ke FPGA untuk membentuk sirkuit.
Flash (board)	Proses menulis bitstream ke FPGA. Pada FALCON memakai <code>xc3sprog</code> . Efek samping: IP interface host hilang & harus di-set ulang.
GTP-U	<i>GPRS Tunneling Protocol — User plane</i> . Protokol yang membungkus (<i>tunnel</i>) data pengguna di jaringan seluler 4G/5G antar elemen inti jaringan.
GTP-C / PFCP / BSSGP	Protokol lain di inti jaringan seluler: GTP-C (kontrol), PFCP (kontrol antar fungsi), BSSGP (jaringan 2G/3G).
DPI	<i>Deep Packet Inspection</i> — memeriksa isi paket secara mendalam (bukan hanya alamat), untuk mengetahui jenis & detail trafik.
Line-rate	Kecepatan penuh kabel jaringan. Memproses "pada <i>line-rate</i> " berarti sanggup mengikuti tanpa tertinggal sama sekali.
TEID	<i>Tunnel Endpoint Identifier</i> — angka unik yang menandai satu "terowongan" (sesi data) GTP-U. Seperti nomor sesi.
IMSI	

Istilah	Pengertian
	<i>International Mobile Subscriber Identity</i> — identitas unik pelanggan seluler (tertanam di SIM). Data sensitif; pada FALCON saat ini disimulasikan/kosong.
QFI	<i>QoS Flow Identifier</i> — penanda kelas kualitas layanan (QoS) suatu aliran data di 5G.
UL / DL	<i>Uplink</i> (dari perangkat ke jaringan) / <i>Downlink</i> (jaringan ke perangkat).
Datagram	Satu paket pesan mandiri. Di FALCON: satu unit telemetri (GLOBAL/TEID/EVENT/PROTOCOL).
UDP	<i>User Datagram Protocol</i> — protokol pengiriman paket yang cepat & ringan tanpa jaminan urutan/keandalan. Cocok untuk telemetri <i>real-time</i> .
WebSocket (WS)	Kanal komunikasi dua arah yang tetap terbuka antara server & browser. FALCON memakainya untuk <i>push</i> data <i>live</i> ke dashboard tanpa <i>refresh</i> .
REST API	Antarmuka berbasis HTTP untuk mengambil data (mis. <code>/api/health</code> , <code>/api/snapshot</code>).
Throttle	Pembatasan laju — FPGA hanya mengirim sebagian sampel telemetri agar tidak membanjiri jaringan host.
Arbiter	Logika yang memilih giliran di antara beberapa sumber yang ingin mengirim pada saat bersamaan.
Serialize / Pack	<i>Pack</i> = menyusun data ke format byte sesuai kontrak. <i>Serialize</i> = mengalirkan byte itu ke kabel.
Kontrak byte (byte contract)	Definisi tepat susunan byte tiap pesan, disepakati & dipakai bersama sumber data dan penerima. <i>Single source of truth</i> FALCON.
Endianness / Big-endian	Urutan penyimpanan byte angka. <i>Big-endian</i> (network order) = byte paling signifikan dulu. Dipakai FALCON.
Header	Bagian awal pesan berisi metadata (di FALCON: tipe, versi, panjang) sebelum isi (<i>payload</i>).
Payload	Isi sebenarnya dari pesan, setelah header.
Ring buffer	Penyangga melingkar berukuran tetap; data terbaru menimpa yang terlama. Dipakai untuk <i>FPGA Telemetry Log</i> (200 entri terakhir).
TTL (Time To Live)	Masa berlaku. Sesi TEID yang tak diperbarui melebihi TTL dianggap mati & dihapus dari tabel.
Desync	Hilangnya sinkronisasi antar bagian sistem. Pada <i>bug</i> utama, ketidakcocokan panjang UDP membuat core UDP <i>desync</i> ~1 detik.
Hardcode	Menuliskan nilai tetap langsung di kode alih-alih menghitungnya. Sumber <i>bug</i> utama (<code>tx_udp_length</code> di-hardcode 68).
Probe / Instrumentasi	Menyisipkan penghitung/penanda sementara ke dalam sirkuit untuk mengamati perilaku internal saat <i>debug</i> .
Combinatorial (kombinatorial)	Logika digital yang keluarannya hanya bergantung pada masukan saat itu (tanpa "ingatan"/register). Perbaikan parser memakai konstanta kombinatorial.
enp2s0	Nama interface Ethernet host yang terhubung ke board FPGA (IP <code>192.168.0.101</code>).
Tailscale	Jaringan privat (VPN mesh) yang membuat perangkat di lokasi berbeda saling terhubung aman seakan satu LAN. Dipakai untuk mengakses dashboard lab dari jarak jauh.
Cloudflare Tunnel	Layanan yang mengekspos aplikasi lokal ke internet publik lewat domain (mis. <code>falcon.arahkarya.com</code>) secara aman.
Simulator	Program yang menghasilkan telemetri tiruan dengan kontrak byte identik FPGA, untuk mengembangkan host tanpa hardware.

Disiapkan oleh Arah Karya Sinergi (AKS) — kolaborasi dengan NOZ.